Assignment #6: 回溯、树、双向链表和哈希表

Updated 1526 GMT+8 Mar 22, 2025

2025 spring, Complied by 物院 李皓翔

> **说明：**
>
> 1. **解题与记录：**
>
>   对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge， Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora https://typoraio.cn 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。
>
> 2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。
>
> 3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。
>
> 请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

LC46.全排列

backtracking, https://leetcode.cn/problems/permutations/

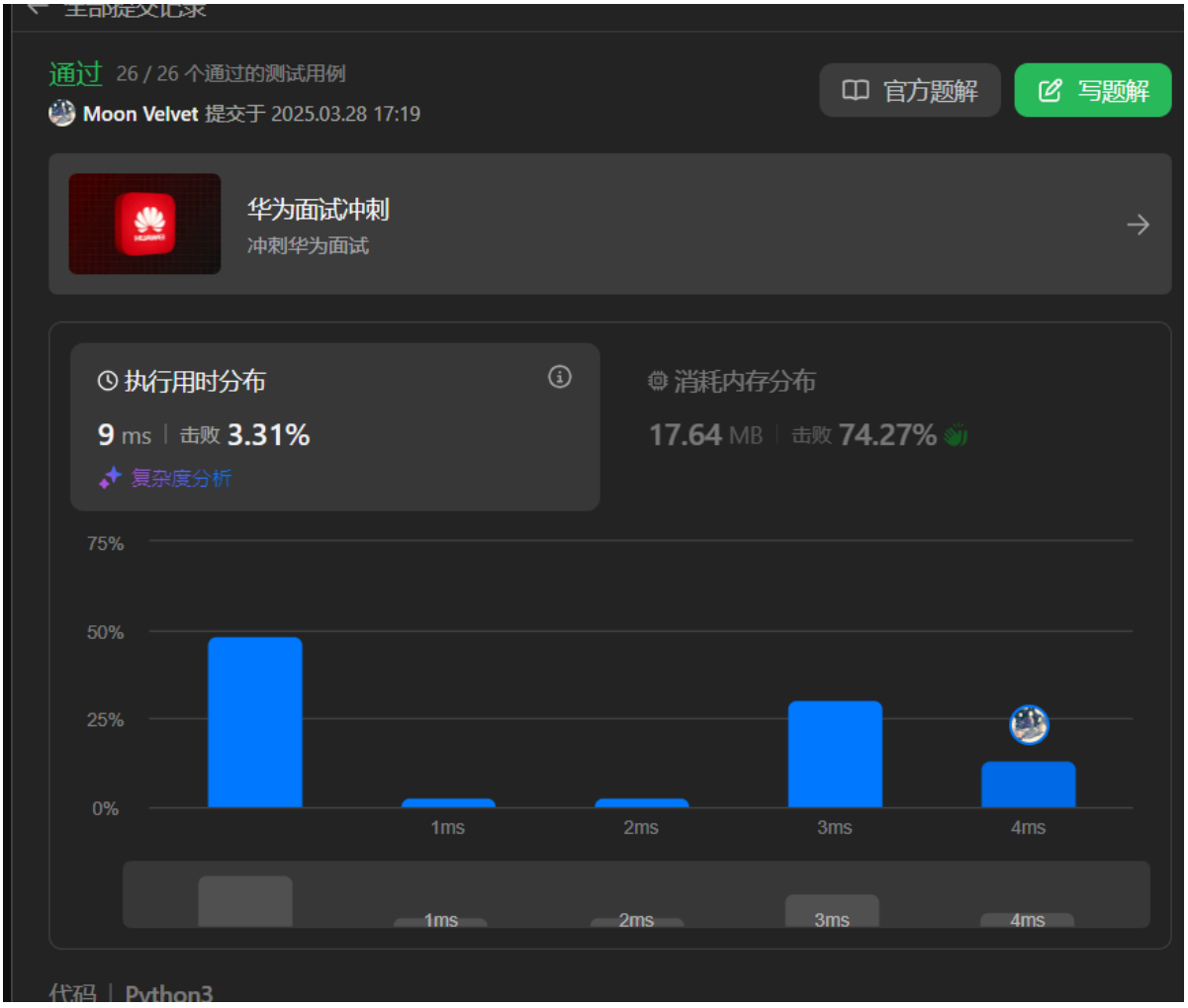
思路：

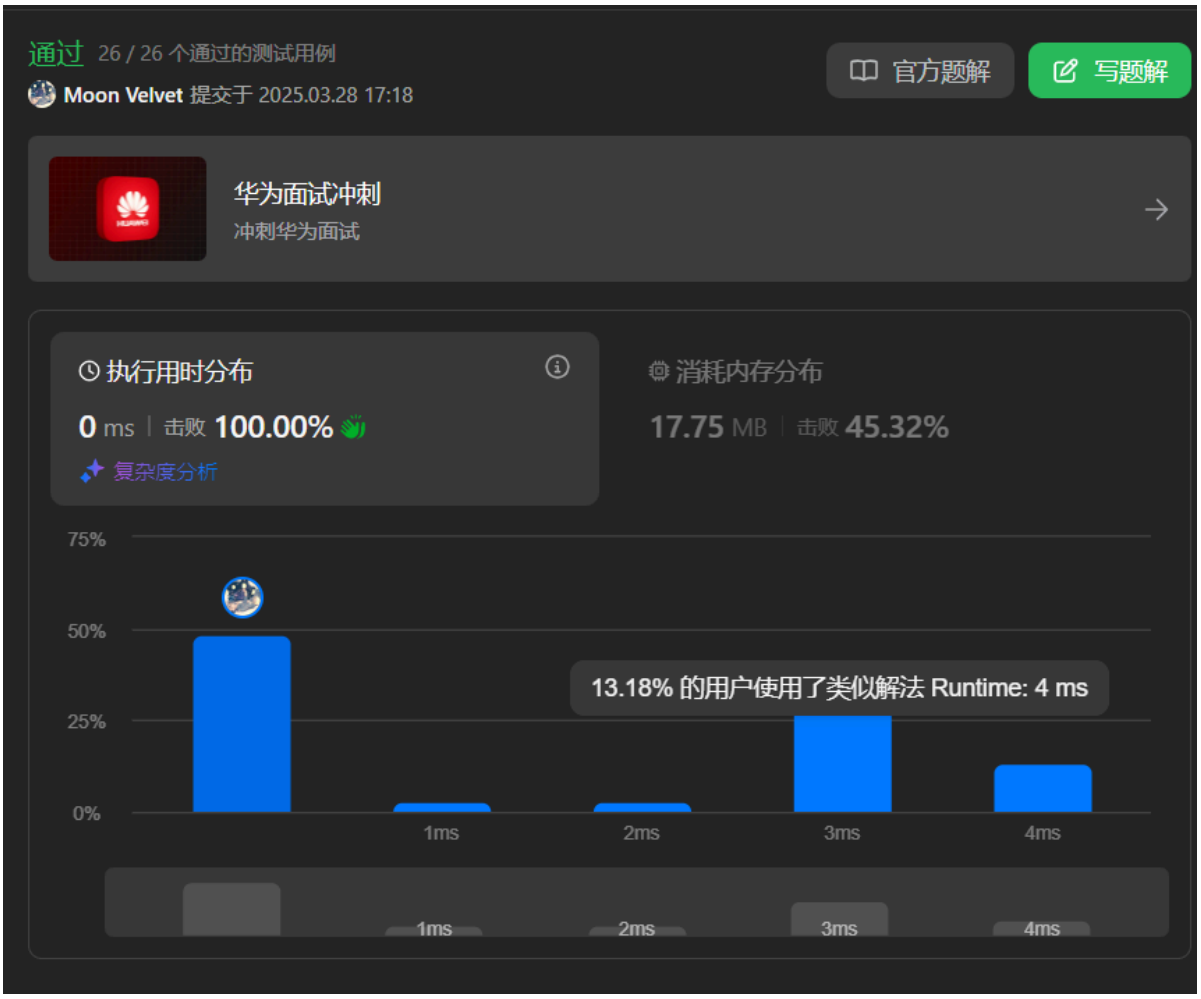
代码：

```
#老解法
from copy import deepcopy
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def dfs(vd,ans,j):
            if len(j)==n:
                tem=deepcopy(j)
                ans.append(tem)
                return ans
            for i in range(n):
                if vd[i]:
                    vd[i]=False
                    j.append(nums[i])
                    ans=dfs(vd,ans,j)
                    j.pop()
```

```
                    vd[i]=True
            return ans
        n=len(nums)
        vd=[True for i in range(n)]
        return dfs(vd,[],[])
#新解法
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        if len(nums) <= 1:
            return [nums]
        ans = []
        for i, num in enumerate(nums):
            n = nums[:i] + nums[i+1: ]
            for y in self.permute(n):
                ans.append([num] + y)
        return ans
```

代码运行截图 （至少包含有"Accepted"）





LC79: 单词搜索

backtracking, https://leetcode.cn/problems/word-search/

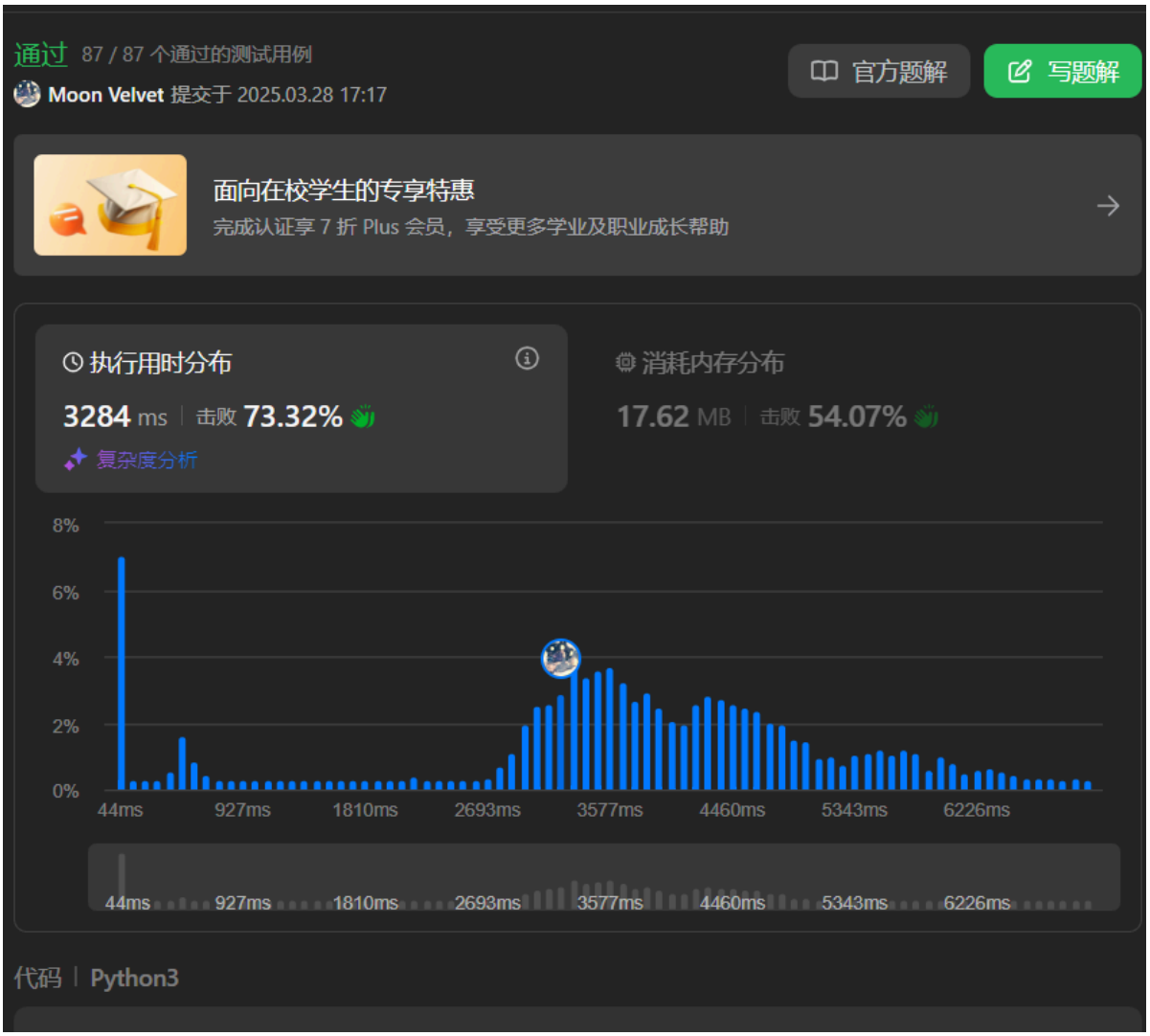
思路:

代码:

```
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        def dfs(vd,k,x,y):
            if k==l:
                return True
            key=word[k]
            vd[x][y]=True
            for i,j in[(1,0),(-1,0),(0,1),(0,-1)]:
                if not(-1<x+i<n and-1<y+j<m):
                    continue
                if key!=board[x+i][y+j] or vd[x+i][y+j]:
                    continue
                if dfs(vd,k+1,x+i,y+j):
                    vd[x][y]=False
                    return True
            vd[x][y]=False
            return False
```

```
        l=len(word)
        n=len(board)
        m=len(board[0])
        vd=[[False for j in range(m)]for i in range(n)]
        for i in range(n):
            for j in range(m):
                if board[i][j]==word[0]:
                    if dfs(vd,1,i,j):
                        return True
        return False
```

代码运行截图 （至少包含有"Accepted"）



LC94.二叉树的中序遍历

dfs, https://leetcode.cn/problems/binary-tree-inorder-traversal/

思路：

代码：

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        a,b=[],[]
        if not root:
            return []
        return self.inorderTraversal(root.left)+
[root.val]+self.inorderTraversal(root.right)
```

代码运行截图 （至少包含有"Accepted"）

LC102.二叉树的层序遍历

bfs, https://leetcode.cn/problems/binary-tree-level-order-traversal/

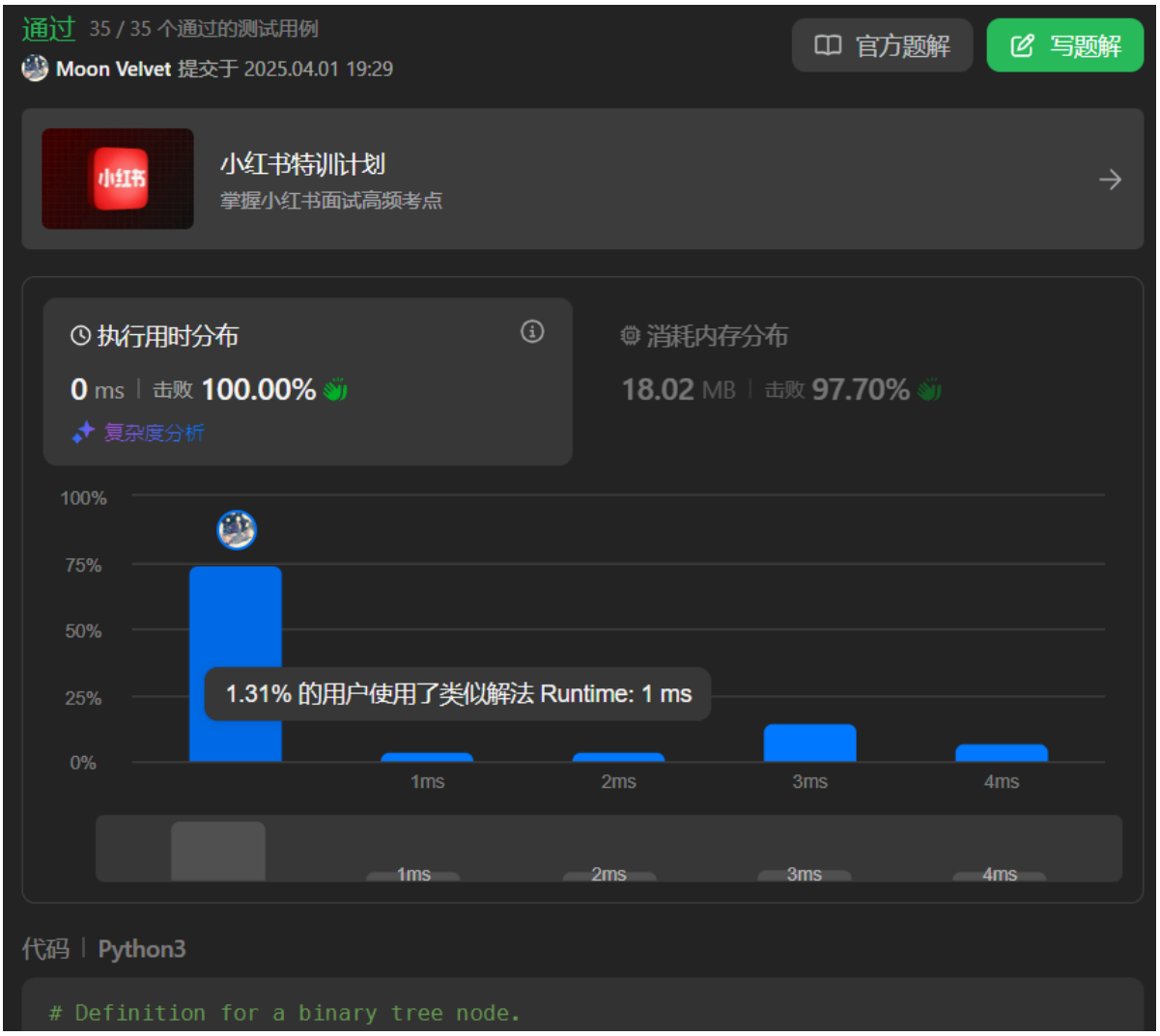
思路：

代码：

```python
# Definition for a binary tree node.
```

```python
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from collections import deque
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []
        stack=deque([root])
        ans=[[]]
        tem=1
        while stack:
            cur=stack.popleft()
            ans[-1].append(cur.val)
            if cur.left:
                stack.append(cur.left)
            if cur.right:
                stack.append(cur.right)
            tem-=1
            if tem==0:
                ans.append([])
                tem=len(stack)
        ans.pop()
        return ans
```

代码运行截图（至少包含有"Accepted"）



LC131.分割回文串

dp, backtracking, https://leetcode.cn/problems/palindrome-partitioning/

思路:

代码:

```
class Solution:
    def partition(self, s: str) -> List[List[str]]:
        n=len(s)
        dp=[[True  for j in range(n)]for i in range(n)]
        for i in range(n-2,-1,-1):
            for j in range(i+1,n):
                dp[i][j]=(s[i]==s[j]) and dp[i+1][j-1]
        ans=[]
        @cache
        def dfs(i):
            if i==n:
                return [[]]
            tem=[]
            for j in range(i,n):
                if dp[i][j]:
```

```
                    for it in dfs(j+1):
                        tem.append([s[i:j+1]]+it)
            return tem
        return dfs(0)
```

代码运行截图 （至少包含有"Accepted"）

LC146.LRU缓存

hash table, doubly-linked list, https://leetcode.cn/problems/lru-cache/

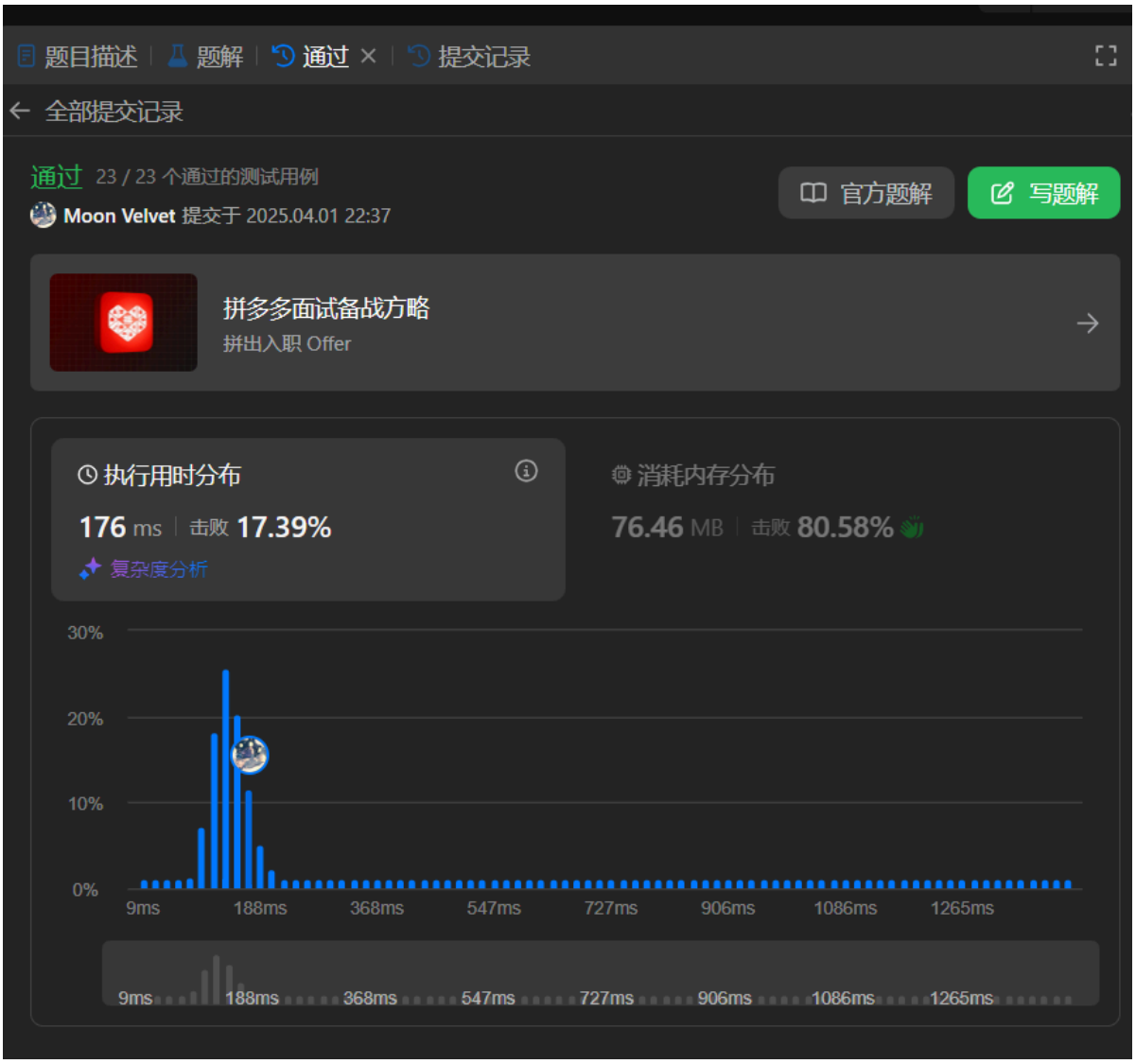
思路：

代码：

```
class DLinkedNode:
    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
```

```
        self.cache={}
        self.head=DLinkedNode()
        self.tail=DLinkedNode()
        self.head.next=self.tail
        self.tail.prev=self.head
        self.capacity=capacity
        self.size=0

    def remove(self,cur):
        cur.prev.next=cur.next
        cur.next.prev=cur.prev

    def addtohead(self,cur):
        cur.next=self.head.next
        cur.prev=self.head
        cur.next.prev=cur
        self.head.next=cur

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        cur=self.cache[key]
        self.remove(cur)
        self.addtohead(cur)
        return cur.value

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            cur=self.cache[key]
            self.remove(cur)
            cur.value=value
            self.addtohead(cur)
        else:
            cur=DLinkedNode(key,value)
            self.cache[key]=cur
            self.addtohead(cur)
            self.size+=1
            if self.size>self.capacity:
                tem=self.tail.prev
                self.remove(tem)
                self.cache.pop(tem.key)
                self.size-=1


# Your LRUCache object will be instantiated and called as such:
# obj = LRUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)
```

代码运行截图 （至少包含有"Accepted"）



2. 学习总结和收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。